

Geobot — Documentación de entrega

Geobot / Agente Educativo — Documentación de entrega (handoff)

Documento para quien asuma el mantenimiento del proyecto. Explica **qué es, dónde está cada cosa, cómo se conectan frontend y backend, cómo se implementa cada LLM**, y la **propuesta del agente inteligente** (varios modelos en uno, según la complejidad de la pregunta).

Si acabas de recibir el proyecto: empieza por la [sección 2 — Guía completa de puesta en marcha](#) (clonar, crear API keys, configurar .env, documentos y npm run dev).

1. Qué es este proyecto (en una frase)

Un MVP de **tutor sobre documentos locales** (PDF, DOCX, XLSX, MD, TXT): el usuario elige una biblioteca y un archivo, hace preguntas, y el sistema responde basándose en el contenido del documento (más extractos de otros archivos del mismo alcance).

No es un agente con tools / function calling. Es un **RAG-lite**:

1. Lee el archivo activo desde disco.
2. Busca por palabras en otros documentos (sin embeddings).
3. Arma un system prompt de “tutor”.
4. Envía todo a un LLM (Gemini, Groq, DeepSeek, etc.).

La idea central del producto es el **Agente inteligente (Smart)**: un solo chat que **elige automáticamente** entre Gemini / Groq / DeepSeek según la pregunta y el tamaño del documento.

2. Stack y guía completa de puesta en marcha

Esta sección es la más importante para quien **recién recibe** el proyecto. Síguela en orden.

2.0 Stack

Capa	Tecnología
Frontend	Next.js 14 (App Router) + React 18 + TypeScript
Backend	Node.js + Express 4 + TypeScript (ESM)
Documentos	Carpeta local en disco (backend/data/ por defecto)
LLMs	Gemini, Groq, DeepSeek, Ollama

Repo: <https://github.com/isabelasring/agente> (rama main).

2.1 Requisitos previos (en tu máquina)

Requisito	Notas
Node.js 18+ (recomendado 20 LTS)	Comprueba con <code>node -v</code> y <code>npm -v</code>
Git	Para clonar el repo
Cuentas cloud para API keys	Al menos Gemini + Groq; DeepSeek recomendado para el Smart completo
(Opcional) Ollama	Solo si vas a probar el panel Ollama local: https://ollama.com

No hace falta Docker ni base de datos: todo corre en local con archivos en disco.

2.2 Qué viene al clonar y qué NO

Ítem	¿Viene al clonar?	Acción
Código frontend/ + backend/src/	Sí	Nada extra
frontend/.env.local	Sí (versionado)	Ya apunta a <code>http://localhost:3001/api/chat</code>
backend/.env.example	Sí	Plantilla sin keys reales
backend/.env	No (.gitignore)	Debes crearlo tú (paso 2.4–2.5)
backend/data/	No (gitignore)	Debes montarla (paso 2.6)

Sin backend/.env con keys → el chat responde *‘Falta configurar ..._API_KEY’*.
Sin backend/data/ → el wizard no muestra bibliotecas.

2.3 Clonar el proyecto

```
git clone https://github.com/isabelasring/agente.git
cd agente
```

(Si usas otro nombre de carpeta, no importa; las rutas relativas backend/ y frontend/ son las que valen.)

2.4 Cómo crear las API keys (paso a paso + enlaces)

El **Agente inteligente** elige entre Gemini, Groq y DeepSeek. Sin esas tres keys el producto queda incompleto (si falta una, esa ruta falla o cae a Gemini).

Regla de oro: pega la key **justo después del =**, sin espacios ni comillas. Mal: GROQ_API_KEY= gsk_xxx · Bien: GROQ_API_KEY=gsk_xxx

Gemini (Google) — obligatoria para Smart (default + fallback)

1. Abre: <https://aistudio.google.com/apikey>
2. Inicia sesión con una cuenta Google.
3. Pulsa **Create API key** (crea o elige un proyecto de Google Cloud si te lo pide).
4. Copia la key (suele empezar por AIza... u otro formato de Google AI).
5. En backend/.env déjala así:

```
GEMINI_API_KEY=pega_aqui_sin_espacios
GEMINI_MODEL=gemini-flash-latest
```

Groq — obligatoria para preguntas cortas/rápidas del Smart

1. Abre: <https://console.groq.com/keys>
(si no tienes cuenta: <https://console.groq.com> → sign up)
2. **Create API Key** → ponle un nombre (ej. geobot) → Create.
3. Copia la key (empieza por gsk_...). **Solo se muestra una vez.**
4. En backend/.env:

```
GROQ_API_KEY=gsk_pega_aqui
GROQ_MODEL=llama-3.1-8b-instant
```

DeepSeek — recomendada para preguntas analíticas / detalladas

1. Abre: https://platform.deepseek.com/api_keys
Registro: <https://platform.deepseek.com>
 2. **Create new API key** → copia el valor (suele empezar por sk-...).
 3. En backend/.env:
- ```
DEEPSEEK_API_KEY=sk-pega_aqui
DEEPSEEK_MODEL=deepseek-v4-flash
```
4. Nota: DeepSeek cobra por uso; necesitas balance/crédito en su plataforma para que responda en producción.

### Ollama — opcional (panel de comparación local, sin API key cloud)

1. Instala Ollama: <https://ollama.com>
  2. En una terminal:
- ```
ollama pull llama3.2
ollama serve
```
3. En backend/.env (valores por defecto ya vienen en .env.example):
- ```
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=llama3.2
```

4. Ollama **no** entra al auto-route del Agente inteligente; solo al panel “Ollama” de la UI.

## Resumen rápido de enlaces

| Proveedor | ¿Para qué en este proyecto?               | Crear / obtener key                                                                 |
|-----------|-------------------------------------------|-------------------------------------------------------------------------------------|
| Gemini    | Smart (default + fallback) + panel Gemini | <a href="https://aistudio.google.com/apikey">aistudio.google.com/apikey</a> .       |
| Groq      | Smart (preguntas cortas) + panel Groq     | <a href="https://console.groq.com/keys">console.groq.com/keys</a>                   |
| DeepSeek  | Smart (analítico) + panel DeepSeek        | <a href="https://platform.deepseek.com/api_keys">platform.deepseek.com/api_keys</a> |
| Ollama    | Solo panel local                          | <a href="https://ollama.com">ollama.com</a> (sin key cloud)                         |

**Mínimo para una demo del Smart:** GEMINI\_API\_KEY + GROQ\_API\_KEY.

**Recomendado en entrega:** esas dos + DEEPSEEK\_API\_KEY.

---

## 2.5 Configurar backend/ .env

Haz esto **antes** de `npm run dev`.

1. Entra al backend y crea el archivo desde la plantilla:

```
cd backend
cp .env.example .env
```

Si en el traspaso te pasaron un `.env` ya armado, cópialo directamente a `backend/.env` (y revisa que las keys sigan válidas).

2. Abre `backend/.env` y complétalo. Ejemplo **completo** de referencia (sustituye los placeholders):

```
PORT=3001
GEOTRENDS_DOCUMENTS_ROOT=data

Gemini — https://aistudio.google.com/apikey
GEMINI_API_KEY=TU_KEY_GEMINI
GEMINI_MODEL=gemini-flash-latest

Groq — https://console.groq.com/keys
GROQ_API_KEY=TU_KEY_GROQ
GROQ_MODEL=llama-3.1-8b-instant
Opcional si Groq responde HTTP 413 con PDFs grandes:
GROQ_MAX_DOCUMENT_CHARS=12000

Ollama (opcional) — https://ollama.com
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=llama3.2

DeepSeek — https://platform.deepseek.com/api_keys
DEEPSEEK_API_KEY=TU_KEY_DEEPSEEK
DEEPSEEK_MODEL=deepseek-v4-flash
```

3. Comprueba estos puntos:

| Check                                         | Por qué importa                                                                         |
|-----------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>PORT=3001</code>                        | Si falta, el backend puede caer a <b>4000</b> y el front (que apunta a 3001) no conecta |
| Keys sin espacios tras =                      | Con espacio, la API rechaza la autenticación                                            |
| Archivo llamado exactamente <code>.env</code> | No <code>.env.txt</code> ni solo editar <code>.env.example</code>                       |
| <b>Nunca</b> subir <code>.env</code> a Git    | Contiene secretos (ya está en <code>.gitignore</code> )                                 |

4. Cada vez que edites `.env`, **reinicia** el backend (`Ctrl+C` y otra vez `npm run dev`) para que cargue los valores nuevos.
- 

## 2.6 Montar la carpeta backend/data/ (documentos)

Esta carpeta **no está en el repo**. Quien entrega el proyecto debe compartirla (ZIP en OneDrive u otro canal).

1. Descarga el ZIP (ej. `geotrends-documentos-data.zip`).
2. Descomprímelo de modo que la ruta final quede **exactamente**:

```
agente/backend/data/
├── Ficha de Servicios/
├── Guía generica/
└── Manuales/
```

3. Comprobación:

```
ls backend/data
Debe listar las bibliotecas (carpetas), no un anidado raro tipo backend/data/data/...
```

4. Si quedó backend/data/data/..., sube el contenido un nivel.
- GEOTRENDS\_DOCUMENTS\_ROOT=data apunta a esa carpeta (relativa al backend).
  - Cada **subcarpeta de primer nivel** = una **Biblioteca** en la UI.
  - Formatos soportados: .pdf, .docx, .xlsx, .md, .txt (no .doc antiguo).

2.7 Frontend .env.local (casi siempre no toques nada)

El archivo **ya viene en el repo**:

```
NEXT_PUBLIC_AGENT_API_URL=http://localhost:3001/api/chat
NEXT_PUBLIC_COURSE_ID=geotrends
NEXT_PUBLIC_LESSON_ID=
```

Solo cámbialo si el backend corre en otro host/puerto. Las keys de LLM **no van aquí**: solo en backend/.env.

2.8 Ejecutar el proyecto (dos terminales)

El frontend espera el backend en **http://localhost:3001**.

Terminal 1 — Backend

```
cd backend
npm install
npm run dev
```

Debe verse algo como: Agent backend running on http://localhost:3001.

Health check: abre o haz GET a <http://localhost:3001/health> → { "ok": true }.

Terminal 2 — Frontend

```
cd frontend
npm install
npm run dev
```

UI: <http://localhost:3000>.

Scripts útiles

| Carpeta   | Comando                   | Uso                                               |
|-----------|---------------------------|---------------------------------------------------|
| backend/  | npm run dev               | Desarrollo (tsx watch, recarga al cambiar código) |
| backend/  | npm run build + npm start | Producción (compila a dist/)                      |
| frontend/ | npm run dev               | Next.js en modo desarrollo                        |
| frontend/ | npm run build + npm start | Next.js producción                                |

2.9 Verificar que todo funciona

1. En <http://localhost:3000>: elige **Biblioteca** → **carpeta/documento** → **documento activo**.
2. Debe aparecer el preview y el bloque **Agente inteligente**.
3. Pregunta corta (ej. *¿de qué trata?*) → suele ir a **Groq** (mira el badge / motivo de routing).
4. Pregunta analítica (ej. *explica en detalle...*) → **DeepSeek** o Gemini.
5. Si ves “Falta configurar GROQ\_API\_KEY...” (u otra): la key no está en .env, está vacía, tiene espacios, o no reiniciaste el backend.

Errores típicos al arrancar

| Síntoma                              | Qué hacer                                                   |
|--------------------------------------|-------------------------------------------------------------|
| “No pude conectar con el backend...” | Backend apagado, o PORT ≠ 3001, o URL mal en .env.local     |
| Wizard sin bibliotecas               | Falta backend/data/ o GEOTRENDS_DOCUMENTS_ROOT              |
| “Falta configurar ..._API_KEY”       | Completa esa key en backend/.env y reinicia backend         |
| Auth rechazada / key inválida        | Regenera la key en la consola del proveedor; quita espacios |
| Groq 413 / fallback a Gemini         | PDF muy grande; prueba GROQ_MAX_DOCUMENT_CHARS=12000        |
| DeepSeek no responde                 | Key o crédito en DeepSeek; si falta key, Smart cae a Gemini |

### 3. Estructura del repositorio

```
agentePrueba/
├── README.md # Resumen + enlace a este handoff
├── DOCUMENTACION-HANDOFF.md # ← este documento (markdown)
├── DOCUMENTACION-HANDOFF.pdf # Misma documentación en PDF
├── frontend/ # UI Next.js (Geobot)
└── backend/ # API Express + lógica LLM
```

Son **dos apps independientes** (sin monorepo npm workspaces ni Docker). Cada una tiene su propio package.json, node\_modules y .env.

**Repo remoto (GitHub):** <https://github.com/isabelasring/agente.git> (rama main).

### 4. Backend — qué hay en cada carpeta

```
backend/
├── package.json # Scripts: dev (tsx watch), build (tsc), start
├── tsconfig.json # strict, module NodeNext, outDir dist/
├── .env / .env.example # Keys y PORT
├── data/ # Corpus documental (bibliotecas)
│ ├── Ficha de Servicios/
│ ├── Guia generica/
│ └── Manuales/
├── src/
│ ├── index.ts # Entry: Express, CORS, /health, monta /api
│ ├── routes/
│ │ └── chat.ts # TODAS las rutas HTTP del producto
│ ├── services/
│ │ ├── context.ts # FS: listar, leer, parsear, cache, path-safe
│ │ ├── search.ts # Búsqueda lexical multi-documento
│ │ ├── llm.ts # Orquestador: askTutor + askSmartTutor
│ │ ├── providerRouter.ts # ★ Propuesta smart: elige Gemini/Groq/DeepSeek
│ │ └── contextBudget.ts # Truncado de contexto por proveedor
│ ├── prompts/
│ │ └── tutorPrompt.ts # System prompt del tutor (rol, fuentes, tono)
│ ├── shared/
│ │ ├── chatHistory.ts # Historial: normalizar, trim (máx 12 msgs), formato OpenAI
│ │ └── llmTypes.ts # AskProviderInput + helpers de error
│ ├── gemini/askGemini.ts # Adapter Gemini (SDK oficial)
│ ├── groq/askGroq.ts # Adapter Groq (fetch OpenAI-compatible + shrink 413)
│ ├── deepseek/askDeepSeek.ts # Adapter DeepSeek (fetch OpenAI-compatible)
│ └── ollama/askOllama.ts # Adapter Ollama local
```

#### 4.1 Entry point — src/index.ts

- Carga dotenv/config.
- cors() abierto + express.json().
- GET /health.
- Monta el router en /api → todo lo de routes/chat.ts queda bajo /api/....
- Puerto: process.env.PORT || 4000 (en .env.example está 3001).

#### 4.2 Rutas — src/routes/chat.ts (contrato HTTP)

| Método | Ruta                                 | Para qué                                |
|--------|--------------------------------------|-----------------------------------------|
| POST   | /api/chat                            | Pregunta al tutor (fijo o auto-route)   |
| GET    | /api/documents?<br>courseId=&folder= | Lista documentos indexables             |
| GET    | /api/folders                         | Bibliotecas top-level                   |
| GET    | /api/folders?parent=                 | Hijos (dirs + files) de una biblioteca  |
| GET    | /api/document?path=                  | Binario (PDF iframe, DOCX, etc.)        |
| GET    | /api/document/text?path=             | Texto extraído (no PDF; máx ~80k chars) |
| GET    | /api/document/html?path=             | HTML tablas XLSX                        |

Body de POST /api/chat

```
{
 "message": "¿De qué trata este documento?",
 "courseId": "geotrends",
 "documentId": "Manuales/algun-pdf.pdf",
 "folder": "Manuales",
 "provider": "auto",
```

```

 "autoRoute": true,
 "history": [
 { "role": "user", "content": "..."},
 { "role": "assistant", "content": "..."}
]
 }
}

```

| Campo                   | Obligatorio | Notas                                                                                     |
|-------------------------|-------------|-------------------------------------------------------------------------------------------|
| message                 | Sí          | Pregunta del usuario                                                                      |
| courseId                | Sí          | Valida el body; hoy el FS <b>no filtra por courseId</b> (legacy). Valor típico: geotrends |
| documentId (o lessonId) | Sí          | Ruta relativa al root documental, o sentinel __geotrends_no_doc__                         |
| folder                  | No          | Prefijo biblioteca/subcarpeta para acotar inventario y búsqueda                           |
| provider                | No          | gemini   groq   deepseek   ollama   auto                                                  |
| autoRoute               | No          | Si true o provider === "auto" → entra al smart router                                     |
| history                 | No          | Solo el Smart lo envía hoy; el backend lo recorta a 12 mensajes                           |

### Respuesta OK

```

{
 "answer": "texto markdown de la respuesta...",
 "meta": {
 "courseId": "geotrends",
 "documentId": "...",
 "provider": "groq",
 "providerLabel": "Groq",
 "autoRoute": true,
 "routingReason": "Pregunta directa o corta → Groq (rapido).",
 "historyMessages": 2,
 "folder": "Manuales",
 "contextLoaded": true,
 "contextChars": 12345,
 "inventorySize": 40,
 "searchHits": [{ "path": "...", "score": 3 }]
 }
}

```

routingReason y providerLabel son lo que muestra el panel Smart en la UI.

### 4.3 Flujo interno de un chat (paso a paso)

Cuando llega POST /api/chat:

1. Valida message + courseId + documentId
2. listCourseDocuments(folder) → inventario
3. Resuelve documentId (si no está en inventario, usa el primero)
4. En paralelo:
  - loadCourseContext → texto completo del documento activo
  - searchAcrossDocuments → hasta 3 snippets de otros archivos
5. Si autoRoute → askSmartTutor(...)
  - Si no → askTutor(..., provider)
6. Responde { answer, meta }

### 4.4 Documentos — services/context.ts

- Root: GEOTRENDS\_DOCUMENTS\_ROOT (relativa al cwd del backend o absoluta). Default conceptual: data.
- Cada **subcarpeta directa** de data/ es una “**Biblioteca**” en la UI (Ficha de Servicios, Guía genérica, Manuales).
- Extensiones indexables: .md, .txt, .docx, .pdf, .xlsx.
- Parseo: PDF (pdf-parse), DOCX (mammoth), XLSX (xlsx), texto plano.
- Cache en memoria por mtimeMs (si el archivo no cambia, no se re-parsea).
- safeResolveUnderRoot: evita path traversal (../ fuera del root).
- Sentinel alineado con el front: \_\_geotrends\_no\_doc\_\_.

### 4.5 Búsqueda — services/search.ts

Búsqueda **lexical** (palabras clave), no vectorial:

- Tokeniza la pregunta, quita stopwords ES/EN.
- Score por frecuencia en el texto + bonus si aparece en el nombre del archivo.

- Devuelve snippets; el prompt los mete como “EXTRACTOS DE OTROS DOCUMENTOS”.

#### 4.6 Prompt del tutor — prompts/tutorPrompt.ts

buildSystemPrompt(...) arma el system message con:

- Rol: analizar docs técnicos/normas sin inventar.
- Orden de fuentes: documento activo → extractos → inventario (solo existencia).
- Instrucciones de formato markdown y tono afirmativo.

Todos los adapters LLM pasan por este prompt.

#### 4.7 Orquestador LLM — services/llm.ts

Dos funciones clave:

|               | Función | Uso                                                                       |
|---------------|---------|---------------------------------------------------------------------------|
| askTutor      |         | Panel fijo: llama al adapter según provider                               |
| askSmartTutor |         | Agente inteligente: pickProvider → budget → ask → posible fallback Gemini |

askTutor hace un switch a:

- askGemini, askGroq, askDeepSeek, askOllama.

Si no hay contexto ni snippets, devuelve un mensaje claro de “no hay contenido cargado...”.

### 5. La propuesta: un agente con varios LLM según complejidad

#### 5.1 Idea de producto

En lugar de forzar al usuario a elegir Gemini vs Groq vs DeepSeek:

1. Un solo chat (**Agente inteligente**).
2. El backend **clasifica** la pregunta (y mira tamaño del doc / historial).
3. Enruta a:
  - **Groq** → rápido, barato, preguntas cortas/directas.
  - **DeepSeek** → analítico, detallado, seguimiento / complejidad.
  - **Gemini** → documentos enormes, default general, y **fallback** si Groq/DeepSeek fallan.

El panel de Ollama (y los fijos Gemini/Groq/DeepSeek) sirve para **comparar proveedores** en demos; el producto “oficial” es el Smart.

#### 5.2 Dónde vive la lógica de decisión

Archivo: **backend/src/services/providerRouter.ts** → función pickProvider.

También usa **contextBudget.ts** → isComplexQuestion (regex de complejidad + historial largo ≥ 10 mensajes).

**Heurísticas de pickProvider (orden aproximado)**

| Condición                                                                              | Proveedor       | Motivo típico      |
|----------------------------------------------------------------------------------------|-----------------|--------------------|
| Pregunta “simple” (regex: título, qué es, fecha...) o corta (< 75 chars) y no compleja | <b>Groq</b>     | Rápido             |
| Pregunta compleja y hay DEEPSEEK_API_KEY                                               | <b>DeepSeek</b> | Analítico          |
| Compleja sin DeepSeek                                                                  | <b>Gemini</b>   | Fallback analítico |
| Documento > 130000 caracteres                                                          | <b>Gemini</b>   | Contexto grande    |
| Pregunta < 130 chars y poco historial                                                  | <b>Groq</b>     | Breve              |
| Pregunta ≥ 130 chars y DeepSeek OK                                                     | <b>DeepSeek</b> | Detallada          |
| Resto                                                                                  | <b>Gemini</b>   | General            |

Regex de “simple” (extracto): titulo|nombre|de que trata|resumen|objeto|cual es|que es|fecha|version|...

Regex de “compleja” (contextBudget.ts): segun lo|compara|articulo|explica en detalle|diferencia entre|calcula|plazo|porque|detalla|fundamenta|requisito|procedimiento|paso a paso|... o historial ≥ 10.

#### 5.3 Presupuesto de contexto por modelo — contextBudget.ts

Antes de llamar al LLM, se recorta el texto del documento:

| Proveedor | Límite aproximado                                          |
|-----------|------------------------------------------------------------|
| Gemini    | 180 000 chars                                              |
| DeepSeek  | 32 000 (normal) / 48 000 (compleja)                        |
| Groq      | 12 000–18 000 (más agresivo; Groq suele devolver HTTP 413) |

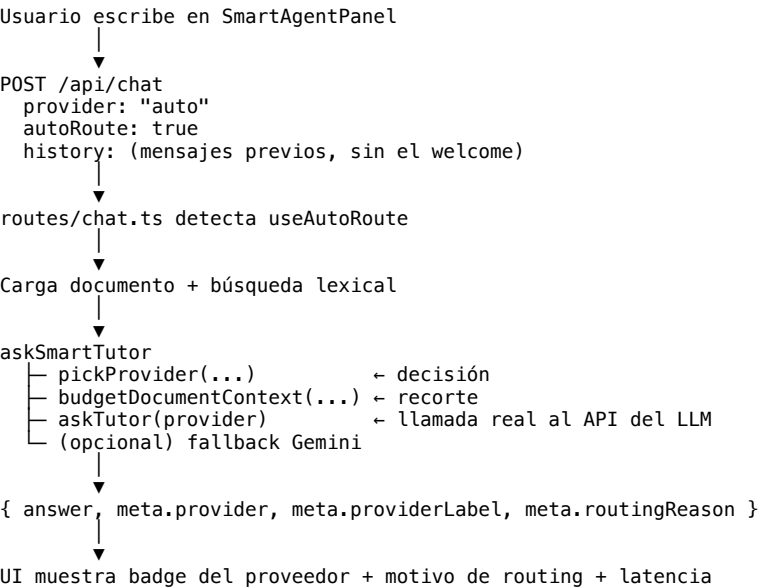
5.4 Fallback a Gemini — askSmartTutor

Si la respuesta de Groq/DeepSeek “parece error” (413, auth, quota, DeepSeek no configurado), se reintenta con Gemini y se concatena en routingReason:

... Fallback → Gemini (groq no disponible o limite).

Eso se ve en la UI bajo “Modelo: ...”.

5.5 Flujo end-to-end del Agente inteligente



5.6 Frontend del Smart — frontend/smart/SmartAgentPanel.tsx

Particularidades vs paneles fijos:

- Envía provider: "auto" y autoRoute: true.
- Envía history (todos los mensajes excepto el welcome).
- El backend limita historial a **12 mensajes** (= hasta ~6 intercambios).
- Lee meta.providerLabel y meta.routingReason y los muestra.
- Shell visual distinto (.smart-agent-shell, color smart).

Los paneles de comparación (GeminiPanel, GroqPanel, ...) envían un provider fijo y **no** mandan historial (cada turno es independiente hacia el API, aunque la UI muestre chat).

6. Cómo se implementa cada LLM (adapters)

Patrón común en todos:

1. Reciben AskProviderInput (message, courseContext, inventario, snippets, history).
2. Construyen system prompt con buildSystemPrompt.
3. Llaman a su API.
4. Devuelven un **string** (respuesta o mensaje de error legible). No lanzan siempre; varios errors se convierten en texto.

| Proveedor | Archivo backend         | Endpoint / SDK real       | Historial | Env keys                        | Modelo default              |
|-----------|-------------------------|---------------------------|-----------|---------------------------------|-----------------------------|
| Gemini    | src/gemini/askGemini.ts | SDK @google/generative-ai | Sí        | GEMINI_API_KEY,<br>GEMINI_MODEL | gemini-<br>flash-<br>latest |

| Proveedor | Archivo backend             | Endpoint / SDK real                                            | Historial | Env keys                                                          | Modelo default       |
|-----------|-----------------------------|----------------------------------------------------------------|-----------|-------------------------------------------------------------------|----------------------|
| Groq      | src/groq/askGroq.ts         | https://api.groq.com/openai/v1/chat/completions                | Sí        | GROQ_API_KEY,<br>GROQ_MODEL, opcional<br>GROQ_MAX_DOCUMENT_CHARS  | llama-3.1-8b-instant |
| DeepSeek  | src/deepseek/askDeepSeek.ts | https://api.deepseek.com/chat/completions                      | Sí        | DEEPSEEK_API_KEY,<br>DEEPSEEK_MODEL, opcional<br>DEEPSEEK_API_URL | deepseek-v4-flash    |
| Ollama    | src/ollama/askOllama.ts     | {OLLAMA_BASE_URL}/api/chat (default<br>http://127.0.0.1:11434) | No        | OLLAMA_BASE_URL,<br>OLLAMA_MODEL                                  | llama3.2             |

Dónde sacar cada API key

Pasos detallados (crear cuenta, copiar, pegar en .env): [sección 2.4](#).

| Proveedor | URL                                                                                         |
|-----------|---------------------------------------------------------------------------------------------|
| Gemini    | <a href="https://aistudio.google.com/apikey">https://aistudio.google.com/apikey</a> .       |
| Groq      | <a href="https://console.groq.com/keys">https://console.groq.com/keys</a>                   |
| DeepSeek  | <a href="https://platform.deepseek.com/api_keys">https://platform.deepseek.com/api_keys</a> |
| Ollama    | <a href="https://ollama.com">https://ollama.com</a> (sin cloud key; ollama pull llama3.2)   |

Notas por proveedor

- Gemini:** modelo por defecto gemini-flash-latest. Es el más tolerante a documentos grandes y el fallback del smart.
- Groq:** si el payload es muy grande → HTTP 413. El adapter hace **retry con shrink progresivo** (recorta doc, inventario, snippets) varias veces. Por eso no conviene mandar PDFs enormes enteros a Groq.
- DeepSeek:** API compatible con Chat Completions; model default en .env.example: deepseek-v4-flash. Si no hay key, el router cae a Gemini en preguntas complejas.
- Ollama:** disponible en panel de comparación; **no entra** al auto-route del Smart.

Panel frontend correspondiente (solo UI; la API real está en backend):

| UI       | Carpeta / archivo                   |
|----------|-------------------------------------|
| Smart    | frontend/smart/SmartAgentPanel.tsx  |
| Gemini   | frontend/gemini/GeminiPanel.tsx     |
| Groq     | frontend/groq/GroqPanel.tsx         |
| DeepSeek | frontend/deepseek/DeepSeekPanel.tsx |
| Ollama   | frontend/ollama/OllamaPanel.tsx     |

7. Frontend — qué hay en cada carpeta

```
frontend/
├── package.json
├── next.config.mjs
├── .env.local # URL del API + courseId
├── app/
│ ├── layout.tsx # Shell HTML + metadata + globals.css
│ ├── page.tsx # * Única página: wizard docs + paneles
│ └── globals.css # Estilos globales (~700 líneas)
├── shared/
│ ├── agentConfig.ts # Código compartido entre paneles
│ ├── agentTypes.ts # URLs, COURSE_ID, sentinels, buildFolderPrefix
│ ├── useCourseDocuments.ts # ChatMessage, DocumentItem, FolderEntry
│ ├── DocumentPreview.tsx # Hooks: docs, carpetas top, hijos biblioteca
│ ├── MessageContent.tsx # Preview PDF / DOCX / texto / XLSX
│ ├── ChatMarkdown.tsx # Assistant → markdown; user → texto
│ └── smart/SmartAgentPanel.tsx # Markdown casero (sin react-markdown)
├── gemini|groq|deepseek|ollama/ # Agente auto-route
├── public/agent-avatar.png
└── agent/ # Paneles por proveedor
 # Carpeta vacía (sin uso)
```

7.1 Página principal — app/page.tsx

Orquesta Geobot:

- Paso 1:** elegir biblioteca (useTopLevelFolders → GET /api/folders).
- Paso 2:** elegir carpeta o archivo dentro (useLibraryChildren).
- Paso 3:** si hizo falta, elegir documento concreto (useCourseDocuments).
- Preview** del documento (DocumentPreview).

- 5. Solo con selección lista:
  - Sección **Agente inteligente** (SmartAgentPanel).
  - Grid **Comparación por proveedor** (6 paneles).

Estado levantado en la página: biblioteca, subcarpeta, selectedDocumentId, etc.

Al cambiar documento/carpeta, los paneles se remontan con `key={...}` → **se reinicia el chat** (intencional).

### 7.2 Config de URLs — shared/agentConfig.ts

`AGENT_CHAT_URL = NEXT_PUBLIC_AGENT_API_URL || "http://localhost:3001/api/chat"`

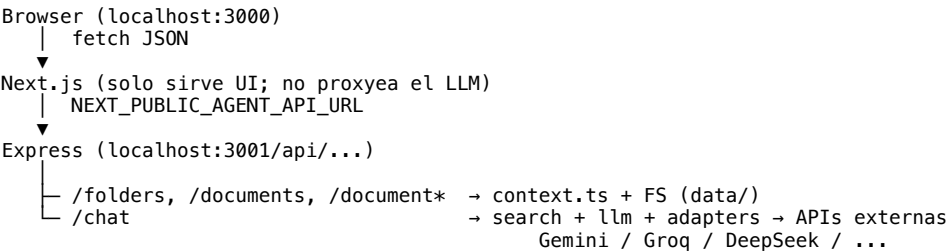
El resto se **deriva** reemplazando `/chat`:

- `/documents, /folders, /document, /document/text, /document/html`

Constantes importantes:

- `COURSE_ID` ← `NEXT_PUBLIC_COURSE_ID` (default `geotrends`)
- `NO_DOCUMENT_SENTINEL` = `__geotrends_no_doc__`
- `buildFolderPrefix(library, subfolder)` → string que el backend usa como `folder`

### 7.3 Cómo se conecta front ↔ back (de punta a punta)



Comunicación: **solo HTTP request/response** con `fetch`. No hay WebSockets ni SSE/streaming.

CORS: el backend tiene `cors()` sin `whitelist` → en local funciona sin más.

### 7.4 Preview de documentos

`DocumentPreview.tsx`:

- PDF → `iframe` a `GET /api/document?path=`
- DOCX → descarga binario + `docx-preview` en el browser
- MD/TXT → `GET /api/document/text`
- XLSX → `GET /api/document/html` (HTML de tablas; se inyecta en el DOM)

## 8. Variables de entorno (backend) — referencia

Plantilla: `backend/.env.example`

Guía paso a paso para crear keys y llenar el archivo: [sección 2.4 y 2.5](#).

| Variable                 | Obligatoria                                     | Para qué                                                   |
|--------------------------|-------------------------------------------------|------------------------------------------------------------|
| PORT                     | Sí (usar <b>3001</b> )                          | Puerto HTTP; alinear con el front                          |
| GEOTRENDS_DOCUMENTS_ROOT | Sí (data)                                       | Carpeta de documentos relativa al backend                  |
| GEMINI_API_KEY           | Sí (Smart)                                      | Auth Gemini — <a href="#">crear key</a>                    |
| GEMINI_MODEL             | No (default <code>gemini-flash-latest</code> )  | Modelo Gemini                                              |
| GROQ_API_KEY             | Sí (Smart rápido)                               | Auth Groq — <a href="#">crear key</a>                      |
| GROQ_MODEL               | No (default <code>llama-3.1-8b-instant</code> ) | Modelo Groq                                                |
| GROQ_MAX_DOCUMENT_CHARS  | No                                              | Techo de chars del doc enviado a Groq (útil ante HTTP 413) |
| DEEPSEEK_API_KEY         | Recomendada                                     | Auth DeepSeek — <a href="#">crear key</a>                  |
| DEEPSEEK_MODEL           | No (default <code>deepseek-v4-flash</code> )    | Modelo DeepSeek                                            |
| DEEPSEEK_API_URL         | No                                              | Override del endpoint Chat Completions                     |
| OLLAMA_BASE_URL          | Solo panel Ollama                               | Default <code>http://127.0.0.1:11434</code>                |
| OLLAMA_MODEL             | Solo panel Ollama                               | Default <code>llama3.2</code>                              |

Para el **Agente inteligente** mínimo operativo:

- 1. GEMINI\_API\_KEY (recomendado siempre: default + fallback)
- 2. GROQ\_API\_KEY (preguntas rápidas)
- 3. DEEPSEEK\_API\_KEY (analítico; si falta, las complejas van a Gemini)

Las keys **nunca** van al frontend: solo viven en backend/.env. Tras editar, reinicia el backend.

## 9. Variables de entorno (frontend)

Archivo: frontend/.env.local

**Estado en el repo:** este archivo **sí está versionado en Git**. Quien clone ya lo recibe. No necesitas crearlo a menos que quieras cambiar la URL del backend.

```
NEXT_PUBLIC_AGENT_API_URL=http://localhost:3001/api/chat
NEXT_PUBLIC_COURSE_ID=geotrends
NEXT_PUBLIC_LESSON_ID=
```

| Variable                  | Efecto                                                    |
|---------------------------|-----------------------------------------------------------|
| NEXT_PUBLIC_AGENT_API_URL | Base del chat; de ahí se derivan las otras URLs           |
| NEXT_PUBLIC_COURSE_ID     | Se manda en cada POST/GET como courseId                   |
| NEXT_PUBLIC_LESSON_ID     | Documento inicial opcional; vacío → sentinel hasta elegir |

## 10. Datos / bibliotecas

Los documentos viven bajo backend/data/ (configurable). Estructura de ejemplo:

```
data/
├── Ficha de Servicios/ ← Biblioteca en la UI
├── Guía generica/ ← Biblioteca
└── Manuales/ ← Biblioteca
```

- Añadir una biblioteca nueva = crear una carpeta de primer nivel dentro del root.
- Añadir documentos = copiar PDF/DOCX/XLSX/MD/TXT dentro (recursivo).
- Reiniciar el backend no es estrictamente necesario para archivos nuevos (se listan al vuelo); el cache se invalida por mtime al cambiar un archivo.

Archivos .doc antiguos (no .docx) **no** se indexan.

## 11. Historial de conversación

| Dónde                    | Comportamiento                                                    |
|--------------------------|-------------------------------------------------------------------|
| Frontend Smart           | Acumula mensajes en estado; envía history sin el welcome          |
| Frontend paneles fijos   | UI de chat, pero el POST normalmente manda solo el mensaje actual |
| Backend chatHistory.ts   | normalizeHistory + trimHistory (máx 12 mensajes)                  |
| Gemini / Groq / DeepSeek | Reciben historial                                                 |
| Ollama                   | No recibe historial hoy                                           |

## 12. Decisiones de diseño (por qué está así)

- 1. **Varios paneles + Smart:** servir para demos/comparación de proveedores, y un modo producto unificado.
- 2. **Routing heurístico (no ML):** reglas simples, predecibles, sin costo extra; fácil de ajustar en providerRouter.ts.
- 3. **Truncado agresivo en Groq:** el proveedor rechaza payloads grandes (413); es mejor recortar que fallar.
- 4. **Sin base de datos:** el corpus es la carpeta; el chat no se persiste (reiniciar página / cambiar doc borra el hilo).
- 5. **Sin auth:** MVP local. Antes de exponer a internet hay que añadir autenticación y rate limit.
- 6. **Sin embeddings:** MVP rápido; mejorar RAG sería el siguiente salto de calidad en respuestas multi-doc.
- 7. **Integración futura Open edX / Vtiger:** la idea era cambiar solo context.ts para traer contexto por API; el front casi no cambia.

## 13. Limitaciones conocidas (importante para el sucesor)

- 1. Sin autenticación ni rate limiting.
- 2. Búsqueda solo lexical (no vectorial / embeddings).
- 3. courseId se valida pero no particiona el corpus en disco.
- 4. Duplicación fuerte entre los 6 paneles del frontend (candidatos a un BaseAgentPanel).
- 5. Sin streaming → UX de espera en preguntas largas.
- 6. Cache de documentos en memoria sin tope.

- 7. Sin tests automatizados.
- 8. Errores HTTP 500 genéricos al cliente.
- 9. Carpeta frontend/agent/ vacía.
- 10. Sin deploy productizado (Docker/CI): solo npm run dev / build+start locales.

## 14. Entrega operativa (lo que suele olvidarse al salir de la empresa)

Esta sección es crítica: el código solo no basta para que el proyecto arranque.

### 14.1 Secretos y archivos que NO vienen en el clone

| Ítem                 | ¿Está en git?   | Qué hacer en la entrega                                                                        |
|----------------------|-----------------|------------------------------------------------------------------------------------------------|
| backend/.env         | No (.gitignore) | Pasarlo por canal seguro (OneDrive privado, chat interno). El sucesor lo pone en backend/.env. |
| backend/data/        | No (gitignore)  | Subir ZIP a OneDrive. El sucesor lo descomprime en backend/data/ (ver sección 2).              |
| frontend/.env.local  | Sí (trackeado)  | <b>No hace falta entregarlo aparte.</b> Ya viene en el clone con la URL local.                 |
| backend/.env.example | Sí              | Plantilla sin secretos.                                                                        |

#### Cómo entregar data/ (quien sale de la empresa)

```
cd backend
zip -r geotrends-documentos-data.zip data
Subir geotrends-documentos-data.zip a OneDrive y compartir el link
```

En el mensaje de traspaso escribe algo como:

- 1. Clona el repo: <https://github.com/isabelasring/agente>
- 2. Sigue la **sección 2** de DOCUMENTACION-HANDOFF.md (crear API keys + .env + data/ + arranque).
- 3. Si te paso un .env ya armado, ponlo en backend/.env (o regenera las keys con los links de la §2.4).
- 4. Descarga el ZIP de documentos y déjalo como backend/data/.
- 5. npm install + npm run dev en backend/ y en frontend/.

### 14.2 Checklist de traspaso humano

- ☐ Transferir ownership / acceso al repo GitHub isabelasring/agente
- ☐ Entregar backend/.env actual (o regenerar keys y actualizar .env)
- ☐ Entregar o ubicar la carpeta backend/data/ con PDFs/docs
- ☐ Indicar quién paga / dueño de cuentas Gemini, Groq, DeepSeek (billing)
- ☐ Probar juntos un chat Smart de punta a punta en la máquina del sucesor
- ☐ (Opcional) Rotar keys después del handoff si la saliente las tenía personales

### 14.3 Problemas frecuentes (troubleshooting)

| Síntoma                                       | Causa habitual                             | Qué revisar                                               |
|-----------------------------------------------|--------------------------------------------|-----------------------------------------------------------|
| Front: “No pude conectar con el backend...”   | Backend apagado o puerto distinto          | npm run dev en backend; PORT=3001; URL en .env.local      |
| Bibliotecas vacías en el wizard               | Falta backend/data/ o root mal configurado | GEOTRENDS_DOCUMENTS_ROOT=data y que existan subcarpetas   |
| “Falta configurar GEMINI_API_KEY...” (u otra) | Key ausente, vacía o con espacios tras =   | backend/.env; ver §2.4–2.5; reiniciar backend             |
| Auth rechazada / key inválida                 | Key rota, mal copiada o con espacio        | Regenerar en la consola del proveedor                     |
| Respuestas de Groq fallan / fallback a Gemini | Payload grande (413) o key inválida        | Logs [smart] / [chat]; GROQ_API_KEY; shrink en askGroq.ts |
| Preguntas complejas siempre a Gemini          | No hay DeepSeek                            | DEEPSEEK_API_KEY en .env                                  |
| Ollama no responde                            | Servicio local no corre                    | ollama serve + modelo OLLAMA_MODEL                        |
| Preview PDF en blanco                         | Ruta inválida o archivo no indexable       | Path relativo correcto; extensión .pdf                    |
| Chat “se borra” al cambiar documento          | Intencional (key en page.tsx)              | No hay persistencia de historial                          |

### 14.4 Regenerar el PDF de esta documentación

Si actualizas el .md y quieres el PDF otra vez:

```
Requiere pandoc + Google Chrome instalados (macOS)
pandoc DOCUMENTACION-HANDOFF.md -t html5 -o /tmp/handoff-geobot.html --standalone \
--metadata title="Geobot - Documentación de entrega"
Abrir el HTML en Chrome → Imprimir → Guardar como PDF
o usar Chrome headless --print-to-pdf=DOCUMENTACION-HANDOFF.pdf
```

## 15. Checklist rápido para el nuevo responsable

- ☐ Acceso al repo [isabelasring/agente](#)
- ☐ Leer **sección 2** completa de este documento
- ☐ Crear keys: [Gemini](#) · [Groq](#) · [DeepSeek](#)
- ☐ cp backend/.env.example backend/.env y pegar las keys **sin espacios** tras =
- ☐ Confirmar PORT=3001 y GEOTRENDS\_DOCUMENTS\_ROOT=data
- ☐ Montar backend/data/ (ZIP del traspaso)
- ☐ Confirmar frontend/.env.local → http://localhost:3001/api/chat
- ☐ npm install + npm run dev en backend/ y frontend/
- ☐ Abrir <http://localhost:3000> y <http://localhost:3001/health>
- ☐ Ver bibliotecas en el wizard (si no, falta data/)
- ☐ Pregunta corta → badge **Groq**; pregunta analítica → **DeepSeek** o Gemini
- ☐ Leer código clave: providerRouter.ts, llm.ts, routes/chat.ts, SmartAgentPanel.tsx

## 16. Mapa “si quiero cambiar X, voy a Y”

| Quiero...                                 | Archivo(s)                                                                                             |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Cambiar reglas del auto-route             | backend/src/services/providerRouter.ts                                                                 |
| Cambiar qué es “pregunta compleja”        | backend/src/services/contextBudget.ts                                                                  |
| Cambiar el tono / instrucciones del tutor | backend/src/prompts/tutorPrompt.ts                                                                     |
| Añadir un LLM nuevo                       | Adapter en backend/src/<nuevo>/, registrar en llm.ts + panel en frontend/<nuevo>/ + opción en page.tsx |
| Meter un LLM al Smart                     | Ampliar RoutedProvider + pickProvider + presupuesto en contextBudget                                   |
| Cambiar endpoints o contrato              | backend/src/routes/chat.ts + frontend/shared/agentConfig.ts                                            |
| Cambiar URLs del API en el front          | frontend/.env.local / agentConfig.ts                                                                   |
| Cambiar carpetas de documentos            | GEOTRENDS_DOCUMENTS_ROOT + contenido de backend/data/                                                  |
| Mejorar preview PDF/DOCX/Excel            | frontend/shared/DocumentPreview.tsx + rutas document en chat.ts                                        |
| Unificar paneles duplicados               | Extraer componente base desde *Panel.tsx                                                               |
| Persistir chats / usuarios                | Hoy no existe: habría que diseñar DB + auth                                                            |

## 17. Diagrama mental del sistema

